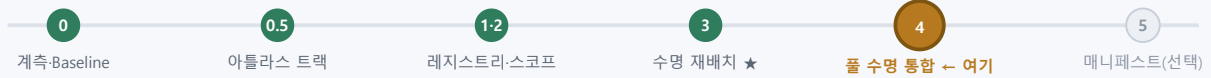


리소스 시스템 Phase 4

풀·인스턴스 수명 통합 (R5) — "에셋을 쓰는 인스턴스"와 "그 에셋 핸들"의 수명을 하나로 묶는다



로드맵. **완료** Phase 0~3 · **다음** Phase 4 (미착수) · **선택** Phase 5. Phase 3가 **에셋 자체**의 수명을 스코프로 잡았다면, Phase 4는 **그 에셋으로 만든 인스턴스 (풀)**의 수명을 같은 체계에 편입한다.

01 한 줄 요약

Phase 3가 리소스를 4스코프로 재배치해 **에셋 핸들**의 수명을 정리했지만, `PoolManager`는 여전히 그 바깥에 있다. 풀은 **인스턴스만 들고 에셋 핸들의 refCount를 잡지 않으며**, 풀 루트가 `DontDestroyOnLoad` (DDOL)라 **썸을 넣어 영생한다**. 즉 "풀 인스턴스의 수명"과 "그 인스턴스가 참조하는 에셋 핸들의 수명"이 서로 다른 관리자에 의해 따로 결정된다 — 지금은 `Clear()` 호출 순서의 우연으로만 사고를 면하고 있다. Phase 4는 **풀을 스코프 소속으로 편입해** 이 불일치를 구조적으로 없앤다.

성격 — 수치가 아니라 안전성. 이 단계는 메모리 그래프를 크게 움직이지 않는다(2.5 예측표: P4 열 = 대부분 불변). Phase 2와 같은 "수치가 안 변해야 성공"인 단계이며, 유일한 신규 성과 지표는 스코프 **Dispose** 시 **잔존 인스턴스 경고 = 0**이다. 값어치는 "Phase 3에서 잡은 use-after-release(`LootNotification` · `ItemSlot` 류)의 마지막 서식지인 풀을 닫는 것".

02 왜 지금 — P4 진단

계획서 §0.2의 **P4(풀↔에셋 수명 미연결)**가 이 단계의 근거다. 현재 코드가 증언한다:

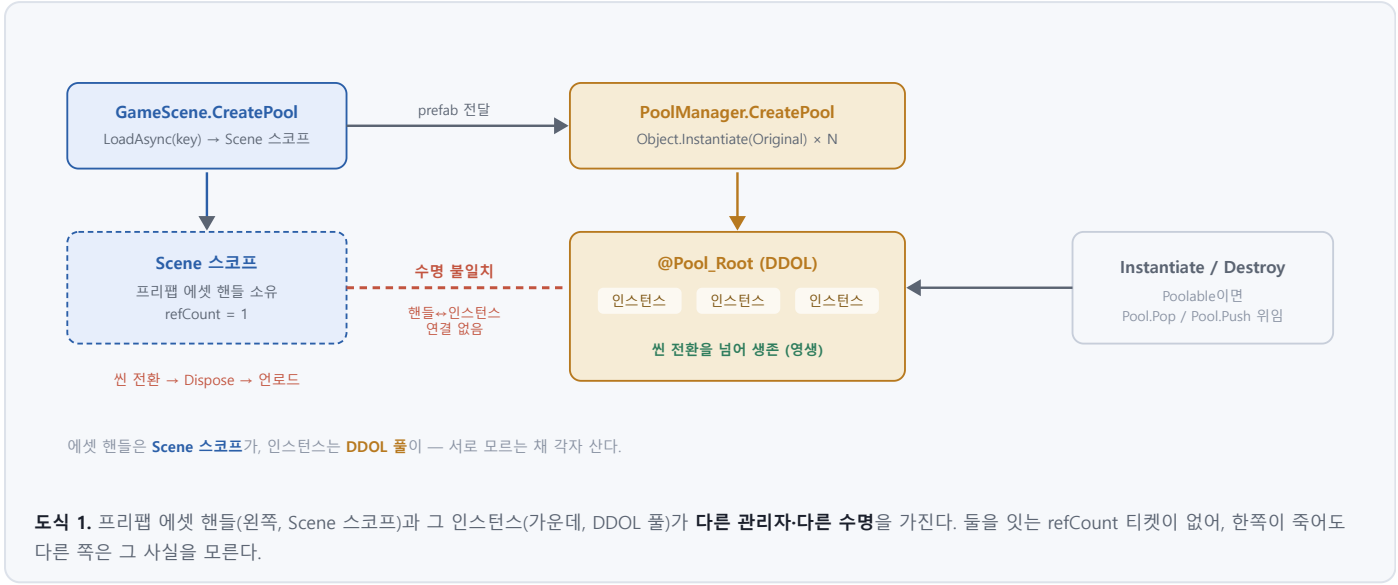
```
// PoolManager.Init() - 루트가 DDOL
_root = new GameObject { name = "@Pool_Root" };
Object.DontDestroyOnLoad(_root); // 썸을 넣어 영생

// Pool.Init(original) - 핸들이 아니라 GameObject만 보관
public GameObject Original { get; }
// _handle = handle; ← 주석 처리됨: 핸들 ref 안 잡음
```

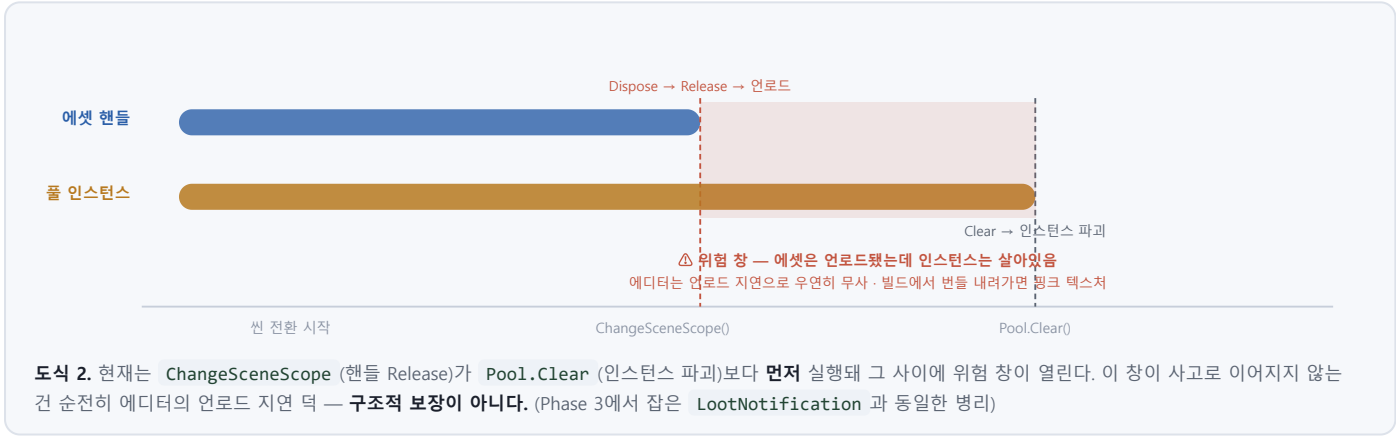
```
// Pool.DestroyPool() - Addressables 반납 없음
// [수정 2] Addressable Release 삭제 및 파괴만 수행
public void DestroyPool() {
    if (Root != null)
        Object.Destroy(Root.gameObject); // 인스턴스만
}
// → 에셋 핸들은 이 경로로 절대 안 풀린다
```

- 풀 인스턴스는 DDOL 루트에 살아 썸 전환을 넘어 생존한다. 그런데 그 인스턴스가 쓰는 프리팹 에셋은 `GameScene.CreatePool` 이 `LoadAsync<GameObject>(key)` = 기본 **Scene** 스코프로 로드한다(Phase 3 이후 `isGlobal` 기본 false).
- 썸 전환 시 `ChangeSceneScope()` 가 Scene 스코프를 Dispose → refCount 0 → **Addressables가 프리팹 에셋을 언로드**. 하지만 풀 인스턴스는 **DDOL에 남는다** = 전형적 **use-after-release**.
- 지금 안 터지는 이유: 썸 전환 경로가 `Managers.Pool.Clear()` 로 인스턴스를 파괴해 우연히 짝을 맞추고, 에디터는 Addressables가 언로드를 지연한다. "**Clear 순서에만 의존해 안 터지는 상태** — 순서 하나 어긋나면 즉시 핑크 텍스처."

03 현재 배선 (AS-IS) — 두 개의 관리자, 두 개의 수명

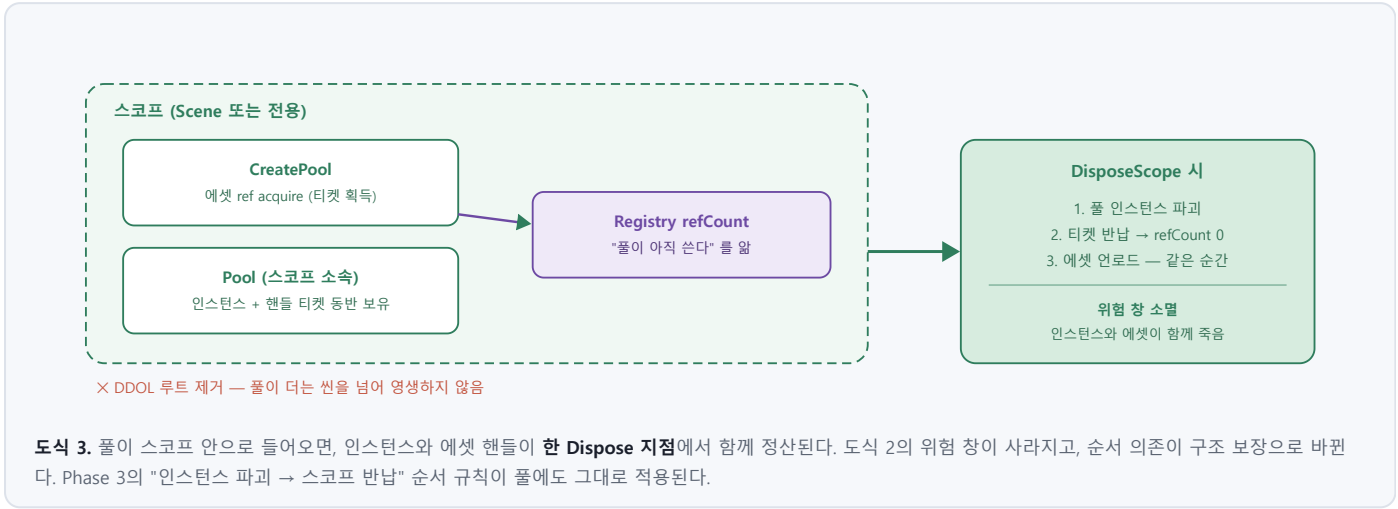


실패 시나리오 — 왜 "순서 하나 어긋나면 터지나"



04 목표 배선 (TO-BE) — 풀을 스코프 소속으로

해법은 Phase 3과 동일한 원리다: 인스턴스를 만든 쪽이 그 에셋의 스코프 티켓을 함께 쥐고, 인스턴스를 버릴 때 티켓도 함께 반납한다. 그러면 refCount가 "이 프리팸을 쓰는 풀이 아직 살아있다"를 알고, 마지막 참조가 사라질 때까지 에셋을 언로드하지 않는다.



05 결가지 — R4 & 완성 조건

Phase 4 창구에 함께 묶인, 리팩터 완주를 위한 잔여 항목이다.

R4 — NoCache / JSON use-after-release 잠재 크래시

원래 Phase 1(P2) 몫이었으나 미착수로 이월됐다. `LoadAsyncNoCache` 가 핸들을 `Release` 한 뒤에 `result`를 반환해, 호출부(JSON 파서)가 이미 언로드될 수 있는 에셋을 참조한다. 현재는 언로드 지연으로 우연히 동작.



남은 완성 조건 (리팩터 전체)

- 개발 빌드 실측 1회 — 에디터 측정치는 CPU 사본 포함 과대 계상이라, 실측 없이는 절대값 주장이 공격받는다. `[ResourceMemory]` 계측 절반이 `CollectDependencies` 의존이므로 런타임 디버그 오버레이 선행 필요.
- 전환 중 인플라이트 낭비의 근본 해결 — "UI 루트가 DDOL이라 씬 UI가 씬보다 오래 사는 구조". Phase 4의 DDOL 루트 제거와 같은 뿌리 — 함께 정리 후보.
- 측정 시나리오 고정 — 히치 지표(현재 n=1~3, "약" 등급)를 등급 상향하려면 세션마다 팝업 오픈 순서·구성을 고정해야 한다.

06 성공 기준 · 예측

이 단계는 대부분의 지표가 **불변이어야 성공**이다(2.5 예측표 P4 열). 신규 항목만 새로 검증한다.

지표	P4 예측	판정
global 핸들 수 / 팝업·던전 증가 / <code>AtlasSpriteCache</code>	불변	변하면 실패 (딴 걸 안 망가뜨림 증명)
진입 직후 Global · 세션 합계	불변	메모리 소폭 — 풀 프리팝이 이미 로드돼 있어 큰 변동 없음
씬 버킷 동작 · 평시 프레임	불변	회귀 없음
스코프 Dispose 시 잔존 인스턴스 경고	0	신규 검증 항목 — 이 단계의 유일한 새 성과 지표

왜 "수치가 안 변하는" 단계에 가치가 있나. Phase 4는 메모리를 더 줄이는 게 아니라, Phase 3에서 세운 스코프 계약을 풀까지 완결하는 단계다. 풀은 use-after-release 병리가 남은 마지막 서식지이고(도식 1~2), 이걸 달아야 "리소스 수명이 전부 스코프 하나로 설명된다"는 리팩터의 최종 명제가 성립한다. Phase 2가 "수치 무관이지만 Phase 3의 전제"였듯, Phase 4는 "수치 무관이지만 안전성의 완결"이다.

상태. Phase 4 미착수 — 다음 세션 재개 지점. 이 문서는 계획이며 측정치는 아직 없다. · 근거 — `Refactor_Plan.md`(§0.2 P4 · §Phase 4 · §2.5 예측표) · `ProblemSolving_Log.md`(R4/R5 항목) · `Phase3_Result.md` §9(남은 작업). 착수 시 코드 진단 대상 — `PoolManager.cs` · `ResourceManager.cs` (Instantiate/Destroy/LoadAsyncNoCache) · `GameScene.cs` (CreatePool).